

- Description

This module manages a smart tunnel air quality monitoring system, ensuring that:

1. **The concentration of harmful gases (CO, NO₂, PM2.5) is continuously measured and compared against safety limits.**
2. **If gas levels exceed the maximum safety threshold, an alert is triggered and ventilation is activated.**
3. **If the air quality improves, the ventilation system automatically reduces power to save energy.**
4. **A log of all air quality readings, alerts, and ventilation adjustments is maintained for environmental compliance.**

This system helps ensure safe air quality in tunnels, reduces pollution risks, and optimizes energy use in ventilation systems.

- Specification

module TunnelAirQualityMonitoringEx

var currentCO, maxCO, currentNO2, maxNO2, currentPM25, maxPM25: real

var ventilationActive: boolean

var log: sequence of string

process updateAirQuality(changeCO: real, changeNO2: real, changePM25: real)

functional scenario form:

currentCO ≤ maxCO && ventilationActive = false, log = log ⊕ ("Air Quality Normal. Ventilation Standby."), Air quality normal, no action needed.

||

currentCO > maxCO && ventilationActive = true, log = log ⊕ ("Air Quality Alert! Ventilation Activated."), Air quality alert, ventilation activated.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.Timer;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.text.DecimalFormat;
import java.util.*;
import java.util.List;
import java.util.concurrent.ThreadLocalRandom;
/**
 * c122_TunnelVentilationSystemEx
 */
```

```

* Single-file implementation with English-only comments.
* SmartWell modules included:
* - Core logic (numeric condition first)
* - Sensors (traffic load, meteo, wind shear, incidents/smoke)
* - Controller (fan staging, jet fans, dampers, purge logic)
* - Protections (over-limit, fast-rise, oscillation, drift, short-
cycle)
* - Logging/CSV
* - Statistics (per pollutant & deltas)
* - Optional Swing GUI
* - CLI test harness (VT/FT/Z3/UVT) - UVT ON by default
*
* Numeric policy reflects urban road tunnels:
* - CO (ppm), NO2 (ppm), PM2.5 (ug/m3)
* - Setpoint banding & near-boundary advisories
* - Power derate and fan short-cycle protection
* - Incident smoke ramp
*
* NOTE: The original public API method is preserved: updateAirQuality(
dCO, dNO2, dPM25).
*     A new updateWithSensors(...) adds fused sensor-driven deltas.
*/
public class c122_TunnelVentilationSystemEx {
    // ===== Harness toggles =====
    private static final boolean RUN_VT      = false;
    private static final boolean RUN_FT      = false;
    private static final boolean RUN_Z3      = false;
    private static final boolean RUN_UVT     = true;
    private static final boolean RUN_RANDOM  = false;
    private static final boolean RUN_GUI     = false;
    private static final boolean VERBOSE_CONSOLE = true;
    public static void main(String[] args) {
        // 1) Policy (numbers-only rules for air quality & actuation)
        VentPolicy policy = new VentPolicy.Builder()
            .setMaxCO(9.0)           // ppm
            .setMaxNO2(1.5)          // ppm
            .setMaxPM25(35.0)        // ug/m3
            .setNearBandCO(1.0)      // advisory band from the
Limit
            .setNearBandNO2(0.2)
            .setNearBandPM25(3.0)
            .setMaxStepCO(80.0)      // max |Δ| per step, anti-
spike
            .setMaxStepNO2(40.0)

```

```

        .setMaxStepPM25(120.0)
        .setShortCycleMinSec(180)    // fan restart minimum seconds
        .setPowerDerate(0.5)         // brownout power derate
        .setSmokeRamp(0.8)           // smoke drives PM2.5 rise factor
        .setOscWindow(8)
        .setDriftWindow(16)
        .setLeakSlopeCO(0.15)
        .setLeakSlopeNO2(0.06)
        .setLeakSlopePM(0.6)
        .setLogCapacity(20000)
        .build();
    // 2) Sensors (synthetic but plausible)
    VentSensorSuite sensors = new VentSensorSuite(
        new TrafficSensor(1200 /*veh/h base*/, 900 /*amplitude*/),
        new MeteoSensor(18.0 /*degC*/, 10.0 /*wind km/h*/, 0.55 /*humidity*/),
        new PortalWindSensor(1.5 /*m/s mean*/, 1.2 /*amp*/),
        new IncidentSensor(0.02 /*smoke prob*/, 0.01 /*fire prob*/),
        new PowerGridSensor(0.02 /*brownout prob*/, 0.01 /*outage prob*/));
    // 3) Controller
    TunnelAirQualityEx aq = new TunnelAirQualityEx(policy, 5.0 /*CO*/, 0.8 /*NO2*/, 20.0 /*PM2.5*/);
    // 4) Demonstration sensor steps (warm-up)
    aq.updateWithSensors(sensors.sample());
    aq.updateWithSensors(sensors.sample());
    aq.updateWithSensors(sensors.sample());
    // 5) Test harness
    // if (RUN_VT) VentTestSuites.runVT(aq);
    // if (RUN_FT) VentTestSuites.runFT(aq);
    // if (RUN_Z3) VentTestSuites.runZ3(aq);
    // if (RUN_UVT) VentTestSuites.runUVT(aq);
    //
    // if (RUN_RANDOM) {
    //     VentRandomSimulator sim = new VentRandomSimulator(sensors);
    //     sim.runSteps(aq, 300); // ~5 minutes at 1s steps for demo
    // }

    VentTestSuites.runVT(aq);

```

```

//      VentTestSuites.runFT(aq);
//      VentTestSuites.runZ3(aq);
//      VentTestSuites.runUVT(aq);
// 6) Print Log (optional)
aq.printLog();
// 7) Statistics
VentStats stats = aq.computeStats();
if (VERBOSE_CONSOLE) {
    System.out.println("\n=== Summary Statistics ===");
    System.out.printf(Locale.US,
        "count=%d | CO mean=%.2f ppm (min=%.2f, max=%.2f, p
95=%.2f)%n",
        stats.count, stats.meanCO, stats.minCO, stats.maxCO
, stats.p95CO);
    System.out.printf(Locale.US,
        "NO2 mean=%.2f ppm (min=%.2f, max=%.2f, p95=%.2f)%n
",
        stats.meanNO2, stats.minNO2, stats.maxNO2, stats.p9
5NO2);
    System.out.printf(Locale.US,
        "PM2.5 mean=%.1f ug/m3 (min=%.1f, max=%.1f, p95=%.1
f)%n",
        stats.meanPM, stats.minPM, stats.maxPM, stats.p95PM
);
    System.out.printf(Locale.US,
        "ΔCO mean=%.2f | ΔNO2 mean=%.2f | ΔPM2.5 mean=%.2f%
n",
        stats.meanDCO, stats.meanDNO2, stats.meanDPM);
}
// 8) CSV export
try {
    String csv = "tunnel_aq_log_c122.csv";
    aq.exportCsv(csv);
    if (VERBOSE_CONSOLE) System.out.println("CSV exported: " +
csv);
} catch (IOException e) {
    System.out.println("CSV export failed: " + e.getMessage());
}
// 9) Optional GUI
if (RUN_GUI) {
    SwingUtilities.invokeLater(() -> {
        VentFrame f = new VentFrame(aq, sensors);
        f.setVisible(true);
    });
}

```

```

    }
}
}
/* =====
 *           Data records & statistics
 * =====
 */
/** Immutable per-step record. */
class AQRecord {
    final long stepIndex;
    final double beforeCO, beforeNO2, beforePM;
    final double dCO, dNO2, dPM;
    final double afterCO, afterNO2, afterPM;
    final boolean ventilationActive;
    final List<String> flags;
    final VentSnapshot snapshot;
    final FanState fanState;
    AQRecord(long stepIndex,
              double beforeCO, double beforeNO2, double beforePM,
              double dCO, double dNO2, double dPM,
              double afterCO, double afterNO2, double afterPM,
              boolean ventilationActive, List<String> flags,
              VentSnapshot snapshot, FanState fanState) {
        this.stepIndex = stepIndex;
        this.beforeCO = beforeCO;
        this.beforeNO2 = beforeNO2;
        this.beforePM = beforePM;
        this.dCO = dCO;
        this.dNO2 = dNO2;
        this.dPM = dPM;
        this.afterCO = afterCO;
        this.afterNO2 = afterNO2;
        this.afterPM = afterPM;
        this.ventilationActive = ventilationActive;
        this.flags = flags;
        this.snapshot = snapshot;
        this.fanState = fanState;
    }
}
/** Simple aggregate statistics across the run. */
class VentStats {
    final int count;
    final double meanCO, minCO, maxCO, p95CO;
    final double meanNO2, minNO2, maxNO2, p95NO2;

```

```

    final double meanPM, minPM, maxPM, p95PM;
    final double meanDCO, meanDNO2, meanDPM;
    VentStats(int count,
               double meanCO, double minCO, double maxCO, double p95CO,
               double meanNO2, double minNO2, double maxNO2, double p95N
02,
               double meanPM, double minPM, double maxPM, double p95PM,
               double meanDCO, double meanDNO2, double meanDPM) {
        this.count = count;
        this.meanCO = meanCO; this.minCO=minCO; this.maxCO=maxCO; this.
p95CO=p95CO;
        this.meanNO2 = meanNO2; this.minNO2=minNO2; this.maxNO2=maxNO2;
        this.p95NO2=p95NO2;
        this.meanPM = meanPM; this.minPM=minPM; this.maxPM=maxPM; this.
p95PM=p95PM;
        this.meanDCO = meanDCO; this.meanDNO2 = meanDNO2; this.meanDPM
= meanDPM;
    }
    static VentStats from(List<AQRecord> recs) {
        if (recs.isEmpty()) {
            return new VentStats(0, Double.NaN, Double.NaN, Double.NaN,
Double.NaN,
                                Double.NaN, Double.NaN, Double.NaN, Double.NaN,
                                Double.NaN, Double.NaN, Double.NaN, Double.NaN,
                                Double.NaN, Double.NaN, Double.NaN);
        }
        List<Double> co = new ArrayList<>(), no2 = new ArrayList<>(), p
m = new ArrayList<>();
        List<Double> dco = new ArrayList<>(), dno2 = new ArrayList<>(),
dpm = new ArrayList<>();
        double sumCO=0, sumNO2=0, sumPM=0, sumDCO=0, sumDNO2=0, sumDPM=
0;
        double minCO=Double.POSITIVE_INFINITY, minNO2=Double.POSITIVE_I
NFINITY, minPM=Double.POSITIVE_INFINITY;
        double maxCO=Double.NEGATIVE_INFINITY, maxNO2=Double.NEGATIVE_I
NFINITY, maxPM=Double.NEGATIVE_INFINITY;
        for (AQRecord r : recs) {
            co.add(r.afterCO); no2.add(r.afterNO2); pm.add(r.afterPM);
            dco.add(r.dCO); dno2.add(r.dNO2); dpm.add(r.dPM);
            sumCO+=r.afterCO; sumNO2+=r.afterNO2; sumPM+=r.afterPM;
            sumDCO+=r.dCO; sumDNO2+=r.dNO2; sumDPM+=r.dPM;
            minCO=Math.min(minCO,r.afterCO); maxCO=Math.max(maxCO,r.aft
erCO);

```

```

        minNO2=Math.min(minNO2,r.afterNO2); maxNO2=Math.max(maxNO2,
r.afterNO2);
        minPM=Math.min(minPM,r.afterPM); maxPM=Math.max(maxPM,r.aft
erPM);
    }
    Collections.sort(co); Collections.sort(no2); Collections.sort(p
m);
    Collections.sort(dco); Collections.sort(dno2); Collections.sort
(dpm);
    int n = recs.size();
    double p95CO = co.get((int)Math.floor(0.95*(n-1)));
    double p95NO2 = no2.get((int)Math.floor(0.95*(n-1)));
    double p95PM = pm.get((int)Math.floor(0.95*(n-1)));
    return new VentStats(n,
        sumCO/n, minCO, maxCO, p95CO,
        sumNO2/n, minNO2, maxNO2, p95NO2,
        sumPM/n, minPM, maxPM, p95PM,
        sumDCO/n, sumDNO2/n, sumDPM/n);
    }
}
/* =====
*
* Policy / limits
* =====
*/
class VentPolicy {
    final double maxCO, maxNO2, maxPM25;
    final double nearBandCO, nearBandNO2, nearBandPM25;
    final double maxStepCO, maxStepNO2, maxStepPM25;
    final int shortCycleMinSec;
    final double powerDerate;
    final double smokeRamp;
    final int oscWindow, driftWindow;
    final double leakSlopeCO, leakSlopeNO2, leakSlopePM;
    final int logCapacity;
    private VentPolicy(Builder b) {
        this.maxCO = b.maxCO; this.maxNO2=b.maxNO2; this.maxPM25=b.maxP
M25;
        this.nearBandCO=b.nearBandCO; this.nearBandNO2=b.nearBandNO2; t
his.nearBandPM25=b.nearBandPM25;
        this.maxStepCO=b.maxStepCO; this.maxStepNO2=b.maxStepNO2; this.
maxStepPM25=b.maxStepPM25;
        this.shortCycleMinSec=b.shortCycleMinSec;
        this.powerDerate=b.powerDerate;
        this.smokeRamp=b.smokeRamp;

```

```

        this.oscWindow=b.oscWindow; this.driftWindow=b.driftWindow;
        this.leakSlopeCO=b.leakSlopeCO; this.leakSlopeNO2=b.leakSlopeNO
2; this.leakSlopePM=b.leakSlopePM;
        this.logCapacity=b.logCapacity;
    }
    boolean nearCO(double v){ return v > (maxCO - nearBandCO); }
    boolean nearNO2(double v){ return v > (maxNO2 - nearBandNO2); }
    boolean nearPM(double v){ return v > (maxPM25 - nearBandPM25); }
    static class Builder {
        private double maxCO=9.0, maxNO2=1.5, maxPM25=35.0;
        private double nearBandCO=1.0, nearBandNO2=0.2, nearBandPM25=3.
0;
        private double maxStepCO=80.0, maxStepNO2=40.0, maxStepPM25=120
.0;
        private int shortCycleMinSec=180;
        private double powerDerate=0.5, smokeRamp=0.8;
        private int oscWindow=8, driftWindow=16;
        private double leakSlopeCO=0.15, leakSlopeNO2=0.06, leakSlopePM
=0.6;
        private int logCapacity=20000;
        Builder setMaxCO(double v){this.maxCO=v;return this;}
        Builder setMaxNO2(double v){this.maxNO2=v;return this;}
        Builder setMaxPM25(double v){this.maxPM25=v;return this;}
        Builder setNearBandCO(double v){this.nearBandCO=v;return this;}
        Builder setNearBandNO2(double v){this.nearBandNO2=v;return this
};}
        Builder setNearBandPM25(double v){this.nearBandPM25=v;return th
is;}
        Builder setMaxStepCO(double v){this.maxStepCO=v;return this;}
        Builder setMaxStepNO2(double v){this.maxStepNO2=v;return this;}
        Builder setMaxStepPM25(double v){this.maxStepPM25=v;return this
};}
        Builder setShortCycleMinSec(int v){this.shortCycleMinSec=v;retu
rn this;}
        Builder setPowerDerate(double v){this.powerDerate=v;return this
};}
        Builder setSmokeRamp(double v){this.smokeRamp=v;return this;}
        Builder setOscWindow(int v){this.oscWindow=v;return this;}
        Builder setDriftWindow(int v){this.driftWindow=v;return this;}
        Builder setLeakSlopeCO(double v){this.leakSlopeCO=v;return this
};}
        Builder setLeakSlopeNO2(double v){this.leakSlopeNO2=v;return th
is;}

```

```

        Builder setLeakSlopePM(double v){this.leakSlopePM=v;return this
;}}

        Builder setLogCapacity(int v){this.logCapacity=v;return this;}
        VentPolicy build(){return new VentPolicy(this);}
    }
}
/* =====
*
*           Sensors & fused snapshot
* =====
*/
enum GridState { NORMAL, BROWNOUT, OUTAGE }
enum IncidentLevel { NONE, SMOKE, FIRE }
enum FanState { OFF, STAGE1, STAGE2, STAGE3, PURGE }
class VentSnapshot {
    final int trafficVehPerHour;
    final double meteoTempC;
    final double externalWindKmH;
    final double humidity; // 0..1
    final double portalWindMS; // Longitudinal portal wind
    final IncidentLevel incident;
    final GridState grid;
    final double suggDCO, suggDNO2, suggDPM; // suggested deltas prior
to protections
    VentSnapshot(int trafficVph, double meteoTempC, double externalWind
KmH, double humidity,
                double portalWindMS, IncidentLevel incident, GridState
grid,
                double dco, double dno2, double dpm) {
        this.trafficVehPerHour = trafficVph;
        this.meteoTempC = meteoTempC;
        this.externalWindKmH = externalWindKmH;
        this.humidity = humidity;
        this.portalWindMS = portalWindMS;
        this.incident = incident;
        this.grid = grid;
        this.suggDCO = dco; this.suggDNO2 = dno2; this.suggDPM = dpm;
    }
}
class TrafficSensor {
    private final int base, amp;
    private int t=0;
    TrafficSensor(int base, int amplitude){this.base=base;this.amp=ampl
itude;}
    int readVPH(){

```

```

        // diurnal wave + noise
        t++;
        double phase = Math.sin((t % 1440) * Math.PI/720.0); // ~24h cycle
        double v = base + amp*phase + ThreadLocalRandom.current().nextGaussian()*60.0;
        v = Math.max(100, v);
        return (int)Math.round(v);
    }
}

class MeteoSensor {
    private final double baseTempC, baseWindKmH;
    private final double baseHum; // 0..1
    private int t=0;
    MeteoSensor(double temp, double wind, double hum){this.baseTempC=temp;this.baseWindKmH=wind;this.baseHum=hum;}
    double tempC(){
        t++;
        return baseTempC + Math.sin(t*Math.PI/720.0)*6.0 + ThreadLocalRandom.current().nextGaussian()*0.5;
    }
    double windKmH(){
        return Math.max(0.0, baseWindKmH + ThreadLocalRandom.current().nextGaussian()*2.0);
    }
    double humidity(){
        double h = baseHum + ThreadLocalRandom.current().nextGaussian()*0.03;
        return Math.max(0.25, Math.min(0.95, h));
    }
}

class PortalWindSensor {
    private final double mean, amp;
    private int t=0;
    PortalWindSensor(double mean, double amp){this.mean=mean;this.amp=amp;}
    double readMS(){
        t++;
        double v = mean + amp*Math.sin(t*Math.PI/90.0) + ThreadLocalRandom.current().nextGaussian()*0.15;
        return Math.max(-3.0, Math.min(3.0, v));
    }
}

class IncidentSensor {

```

```

    private final double smokeProb, fireProb;
    private final Random rnd = new Random(1221);
    private int smokeRemain=0, fireRemain=0;
    IncidentSensor(double smokeProb, double fireProb){this.smokeProb=sm
okeProb;this.fireProb=fireProb;}
    IncidentLevel read(){
        if (fireRemain>0){fireRemain--;return IncidentLevel.FIRE;}
        if (smokeRemain>0){smokeRemain--;return IncidentLevel.SMOKE;}
        double r = rnd.nextDouble();
        if (r < fireProb) { fireRemain = 120 + rnd.nextInt(120); return
IncidentLevel.FIRE; }
        if (r < fireProb + smokeProb) { smokeRemain = 90 + rnd.nextInt(
120); return IncidentLevel.SMOKE; }
        return IncidentLevel.NONE;
    }
}
class PowerGridSensor {
    private final double brownoutProb, outageProb;
    private final Random rnd = new Random(1222);
    PowerGridSensor(double brownoutProb, double outageProb){this.browno
utProb=brownoutProb;this.outageProb=outageProb;}
    GridState read(){
        double r = rnd.nextDouble();
        if (r < outageProb) return GridState.OUTAGE;
        if (r < outageProb + brownoutProb) return GridState.BROWNOUT;
        return GridState.NORMAL;
    }
}
/** Sensor suite + fusion to propose pollutant deltas based on traffic
& incidents. */
class VentSensorSuite {
    private final TrafficSensor traffic;
    private final MeteoSensor meteo;
    private final PortalWindSensor portal;
    private final IncidentSensor incidents;
    private final PowerGridSensor grid;
    VentSensorSuite(TrafficSensor t, MeteoSensor m, PortalWindSensor p,
IncidentSensor i, PowerGridSensor g){
        this.traffic=t; this.meteo=m; this.portal=p; this.incidents=i;
this.grid=g;
    }
    VentSnapshot sample(){
        int vph = traffic.readVPH();
        double temp = meteo.tempC();

```

```

double wind = meteo.windKmH();
double hum  = meteo.humidity();
double pw   = portal.readMS();
IncidentLevel inc = incidents.read();
GridState gs = grid.read();
// Very simple emission model (numbers-first, not domain-
calibrated):
    // - CO ~ traffic intensity, attenuated by longitudinal portal
wind
    // - NO2 ~ traffic & temperature (hot engines), moist air slight-
ly increases NO2 persistence
    // - PM2.5 ~ traffic + smoke/fire incident (strong)
double baseCO  = 0.0045 * vph;           // ppm per step
double baseNO2 = 0.0009 * vph + 0.01*(temp-18.0) + 0.05*(hum-
0.5);
double basePM  = 0.06 * (vph/100.0);     // ug/m3 per step
// Portal wind helps sweep: negative wind reduces build-
up; clamp magnitude
baseCO  += -0.9 * Math.signum(pw) * Math.min(1.2, Math.abs(pw));
baseNO2 += -
0.35 * Math.signum(pw) * Math.min(1.2, Math.abs(pw));
basePM  += -1.8 * Math.signum(pw) * Math.min(1.2, Math.abs(pw));
// Incident smoke/fire adds PM (and some CO/NO2)
if (inc == IncidentLevel.SMOKE) {
    basePM += 6.5;
    baseCO += 0.7;
    baseNO2 += 0.2;
} else if (inc == IncidentLevel.FIRE) {
    basePM += 16.0;
    baseCO += 1.6;
    baseNO2 += 0.5;
}
// Brownout/outage slightly reduces emissions but also reduces
ventilation capacity (handled later)
if (gs == GridState.OUTAGE) {
    baseCO *= 0.95; baseNO2 *= 0.95; basePM *= 0.95;
} else if (gs == GridState.BROWNOUT) {
    baseCO *= 0.98; baseNO2 *= 0.98; basePM *= 0.98;
}
// Add bounded random noise
baseCO += ThreadLocalRandom.current().nextGaussian()*0.2;
baseNO2 += ThreadLocalRandom.current().nextGaussian()*0.05;
basePM  += ThreadLocalRandom.current().nextGaussian()*1.0;

```

```

        return new VentSnapshot(vph, temp, wind, hum, pw, inc, gs, base
CO, baseNO2, basePM);
    }
}
/* =====
*
*           Anomaly detection
* =====
*/
class VentAnomalyDetector {
    static List<String> detect(VentPolicy p,
                                double beforeCO, double beforeNO2, doubl
e beforePM,
                                double dCO, double dNO2, double dPM,
                                double afterCO, double afterNO2, double
afterPM,
                                List<AQRecord> hist, VentSnapshot s,
                                FanState fanState, int secSinceFanOn, in
t secSinceFanOff) {
    List<String> flags = new ArrayList<>();
    // Step clamps
    if (Math.abs(dCO) > p.maxStepCO) flags.add("CO step exceeds cl
amp");
    if (Math.abs(dNO2) > p.maxStepNO2) flags.add("NO2 step exceeds
clamp");
    if (Math.abs(dPM) > p.maxStepPM25) flags.add("PM2.5 step exceed
s clamp");
    // Basic bounds (over-limit alerts)
    if (afterCO > p.maxCO) flags.add("CO above limit (ALERT)");
    else if (p.nearCO(afterCO)) flags.add("CO near upper limit (Adv
isory)");
    if (afterNO2 > p.maxNO2) flags.add("NO2 above limit (ALERT)");
    else if (p.nearNO2(afterNO2)) flags.add("NO2 near upper limit (
Advisory)");
    if (afterPM > p.maxPM25) flags.add("PM2.5 above limit (ALERT)");
    ;
    else if (p.nearPM(afterPM)) flags.add("PM2.5 near upper limit (
Advisory)");
    // Grid/incident flags
    if (s != null) {
        if (s.grid == GridState.BROWNOUT) flags.add("Grid brownout:
ventilation derated");
        if (s.grid == GridState.OUTAGE) flags.add("Grid outage: ac
tive ventilation unavailable");
    }
}

```

```

        if (s.incident == IncidentLevel.SMOKE) flags.add("Incident:
SMOKE reported");
        if (s.incident == IncidentLevel.FIRE) flags.add("Incident:
FIRE reported");
    }
    // Short-cycle / fan state info
    if (fanState != FanState.OFF && secSinceFanOn < 3) {
        flags.add("Fans just engaged");
    }
    if (fanState == FanState.OFF && secSinceFanOff < p.shortCycleMi
nSec) {
        flags.add("Short-cycle protection active");
    }
    // Oscillations (sign flip with magnitude)
    if (hist.size() >= p.oscWindow) {
        if (isOscillating(hist, p.oscWindow)) flags.add("Oscillatio
n pattern detected");
    }
    // Drift / Leak (upward trends)
    if (hist.size() >= p.driftWindow) {
        double slopeCO = estimateSlope(hist, p.driftWindow, 'C');
        double slopeNO2= estimateSlope(hist, p.driftWindow, 'N');
        double slopePM = estimateSlope(hist, p.driftWindow, 'P');
        if (slopeCO > p.leakSlopeCO) flags.add("CO upward drift");
        if (slopeNO2 > p.leakSlopeNO2) flags.add("NO2 upward drift"
);
        if (slopePM > p.leakSlopePM) flags.add("PM2.5 upward drift"
);
    }
    return flags;
}

private static boolean isOscillating(List<AQRecord> hist, int w) {
    int n = hist.size();
    int flips = 0;
    for (int i=n-w+1; i<n; i++) {
        double s1 = Math.signum(hist.get(i-1).dCO + hist.get(i-
1).dNO2 + hist.get(i-1).dPM);
        double s2 = Math.signum(hist.get(i).dCO + hist.get(i).dNO2
+ hist.get(i).dPM);
        if (s1 != s2 && Math.abs(hist.get(i-
1).dPM) > 10 && Math.abs(hist.get(i).dPM) > 10) flips++;
    }
    return flips >= (w/2);
}

```

```

    private static double estimateSlope(List<AQRecord> hist, int k, char tag) {
        int n = Math.min(k, hist.size());
        double sumX=0, sumY=0, sumXY=0, sumX2=0;
        for (int i=0;i<n;i++){
            double x=i, y;
            AQRecord r = hist.get(hist.size()-1-i);
            if (tag=='C') y=r.afterCO;
            else if (tag=='N') y=r.afterNO2;
            else y=r.afterPM;
            sumX+=x; sumY+=y; sumXY+=x*y; sumX2+=x*x;
        }
        double denom = n*sumX2 - sumX*sumX;
        if (denom==0) return 0.0;
        return (n*sumXY - sumX*sumY)/denom;
    }
}

/* =====
 *
 *           Core Controller
 * =====
 */
class TunnelAirQualityEx {
    private final VentPolicy policy;
    private double co, no2, pm;           // current levels
    private boolean ventilationActive=false;
    private final CircularBuffer122<AQRecord> log;
    private long step=0;
    // fan state and simple short-cycle timing
    private FanState fanState = FanState.OFF;
    private int secondsSinceFanOn = 9999;
    private int secondsSinceFanOff = 9999;
    TunnelAirQualityEx(VentPolicy policy, double initialCO, double initialNO2, double initialPM25) {
        this.policy = policy;
        this.co = initialCO; this.no2=initialNO2; this.pm=initialPM25;
        this.log = new CircularBuffer122<>(policy.logCapacity);
    }
    /** Original API: update by arbitrary deltas (could be test sequences). */
    public void updateAirQuality(double changeCO, double changeNO2, double changePM25) {
        updateInternal(changeCO, changeNO2, changePM25, null);
    }
}

```

```

    /** Sensor-
driven update: fuse, protect, and apply ventilation response. */
    public void updateWithSensors(VentSnapshot s) {
        // Start from sensor suggestion
        double dCO = s.suggDCO;
        double dNO2 = s.suggDN02;
        double dPM = s.suggDPM;
        // Grid state reduces effect of ventilation (handled during act
uation),
        // but here we emulate a small passive benefit if wind aligns n
egative:
        if (s.grid == GridState.OUTAGE) {
            // no active ventilation; pollutants drift up if traffic is
heavy
            dCO += 0.1; dNO2 += 0.04; dPM += policy.smokeRamp; // mild
drift
        } else if (s.grid == GridState.BROWNOUT) {
            // partial effect remains
            dCO *= 1.02; dNO2 *= 1.02; dPM *= 1.02;
        }
        // Determine staging by comparing to limits
        FanState desired = computeDesiredFanState(s);
        applyShortCycleProtection(desired);
        desired = this.fanState; // after protection
        // Translate fan state into negative deltas (removal), subject
to grid derate
        double ventFactor = fanVentilationFactor(desired);
        if (s.grid == GridState.BROWNOUT) ventFactor *= policy.powerDer
ate;
        if (s.grid == GridState.OUTAGE) ventFactor = 0.0;
        // Purge gives stronger PM removal especially during smoke/fire
        double pmBias = (s.incident==IncidentLevel.FIRE) ? 3.5 : (s.inc
ident==IncidentLevel.SMOKE? 1.8 : 1.0);
        dCO -= 0.65 * ventFactor;
        dNO2 -= 0.22 * ventFactor;
        dPM -= (1.6 * ventFactor * pmBias);
        updateInternal(dCO, dNO2, dPM, s);
        secondsSinceFanOn++;
        secondsSinceFanOff++;
    }
    private void applyShortCycleProtection(FanState desired) {
        // Prevent immediate re-start after shutdown
        if (desired != FanState.OFF && fanState == FanState.OFF && seco
ndsSinceFanOff < policy.shortCycleMinSec) {

```

```

        // delay restart, keep OFF
        return;
    }
    if (desired == FanState.OFF && fanState != FanState.OFF) {
        fanState = FanState.OFF;
        secondsSinceFanOff = 0;
        return;
    }
    if (desired != fanState) {
        fanState = desired;
        secondsSinceFanOn = 0;
    }
}

private FanState computeDesiredFanState(VentSnapshot s) {
    // Rule-based staging (numbers-first)
    boolean overCO = co > policy.maxCO;
    boolean overNO2 = no2 > policy.maxNO2;
    boolean overPM = pm > policy.maxPM25;
    boolean nearCO = policy.nearCO(co);
    boolean nearNO2 = policy.nearNO2(no2);
    boolean nearPM = policy.nearPM(pm);
    // Incident-driven purge
    if (s.incident == IncidentLevel.FIRE) return FanState.PURGE;
    if (s.incident == IncidentLevel.SMOKE && (overPM || nearPM)) re
turn FanState.PURGE;
    // Over-limit triggers
    int overs = (overCO?1:0) + (overNO2?1:0) + (overPM?1:0);
    if (overs >= 2) return FanState.STAGE3;
    if (overCO || overNO2 || overPM) return FanState.STAGE2;
    // Near-boundary advisories
    int nears = (nearCO?1:0)+(nearNO2?1:0)+(nearPM?1:0);
    if (nears >= 2) return FanState.STAGE2;
    if (nears == 1) return FanState.STAGE1;
    // Traffic surge heuristic
    if (s.trafficVehPerHour > 2200) return FanState.STAGE1;
    return FanState.OFF;
}

private double fanVentilationFactor(FanState fs) {
    switch (fs){
        case OFF: return 0.0;
        case STAGE1: return 0.8;
        case STAGE2: return 1.6;
        case STAGE3: return 2.5;
        case PURGE: return 4.0;
    }
}

```

```

        default:    return 0.0;
    }
}

private void updateInternal(double dCO, double dNO2, double dPM, VentSnapshot s) {
    // Clamp steps
    dCO = clampStep(dCO, policy.maxStepCO);
    dNO2 = clampStep(dNO2, policy.maxStepNO2);
    dPM = clampStep(dPM, policy.maxStepPM25);
    double beforeCO = co, beforeNO2 = no2, beforePM = pm;
    double afterCO = beforeCO + dCO;
    double afterNO2 = beforeNO2 + dNO2;
    double afterPM = beforePM + dPM;
    // Hard safety clamps to keep within plausible extremes (not real-world limits)
    afterCO = clamp(afterCO, -10.0, 1000.0);
    afterNO2 = clamp(afterNO2, -2.0, 200.0);
    afterPM = clamp(afterPM, -10.0, 5000.0);
    // Update ventilationActive flag for message
    ventilationActive = (fanState != FanState.OFF);
    // Build flags
    List<AQRecord> history = log.snapshotView();
    List<String> flags = VentAnomalyDetector.detect(policy,
        beforeCO, beforeNO2, beforePM,
        dCO, dNO2, dPM,
        afterCO, afterNO2, afterPM,
        history, s, fanState, secondsSinceFanOn, secondsSinceFanOff);
    // Compose message
    String msg = String.format(Locale.US,
        "Step #%d | CO: %.2f ppm (Δ%+, .2f) | NO2: %.2f ppm (Δ%+, .2f) | PM2.5: %.1f ug/m³ (Δ%+, .1f) | Fan=%s",
        ++step, afterCO, dCO, afterNO2, dNO2, afterPM, dPM, fanState.name());
    if (!flags.isEmpty()) msg += " | " + String.join("; ", flags);
    System.out.println(msg);
    // Commit
    co=afterCO; no2=afterNO2; pm=afterPM;
    // Snapshot fallback if null
    VentSnapshot snap = (s!=null)? s :
        new VentSnapshot(0, Double.NaN, Double.NaN, Double.NaN,
            Double.NaN,
            IncidentLevel.NONE, GridState.NORMAL, Double.NaN,
            Double.NaN, Double.NaN);
}

```

```

AQRecord rec = new AQRecord(step, beforeCO, beforeNO2, beforePM
,
    dCO, dNO2, dPM, afterCO, afterNO2, afterPM,
    ventilationActive, flags, snap, fanState);
log.add(rec);
}
private static double clampStep(double d, double maxAbs) {
    if (d > maxAbs) return maxAbs;
    if (d < -maxAbs) return -maxAbs;
    return d;
}
private static double clamp(double v, double lo, double hi){
    return Math.max(lo, Math.min(hi, v));
}
public void printLog() {
    System.out.println("\nTunnel Air Quality Log:");
    for (AQRecord r : log.iterable()) {
        String line = String.format(Locale.US,
            "Step %d | CO=%.2f NO2=%.2f PM=%.1f | Fan=%s | Flag
s=%s",
            r.stepIndex, r.afterCO, r.afterNO2, r.afterPM,
            r.fanState.name(), r.flags.isEmpty()? "-"
: String.join("|", r.flags));
        System.out.println(line);
    }
}
public VentStats computeStats() {
    return VentStats.from(log.asList());
}
public void exportCsv(String file) throws IOException {
    try (PrintWriter pw = new PrintWriter(new BufferedWriter(new Fi
leWriter(file)))) {
        pw.println("step,beforeCO,beforeNO2,beforePM,dCO,dNO2,dPM,a
fterCO,afterNO2,afterPM,ventilationActive,flags," +
            "trafficVPH,meteoTempC,externalWindKmH,humidity,por
talWindMS,incident,grid,fan");
        for (AQRecord r : log.iterable()) {
            String flags = String.join("|", r.flags);
            pw.printf(Locale.US,
                "%d,%.6f,%.6f,%.6f,%.6f,%.6f,%.6f,%.6f,%.6
f,%.6f,%b,%s,%d,%.6f,%.6f,%.6f,%.6f,%s,%s,%s%n",
                r.stepIndex, r.beforeCO, r.beforeNO2, r.beforeP
M,

```

```

        r.dCO, r.dNO2, r.dPM, r.afterCO, r.afterNO2, r.
afterPM,
        r.ventilationActive, csvEscape(flags),
        r.snapshot.trafficVehPerHour, r.snapshot.meteoT
empC, r.snapshot.externalWindKmH,
        r.snapshot.humidity, r.snapshot.portalWindMS,
        r.snapshot.incident.name(), r.snapshot.grid.nam
e(), r.fanState.name());
    }
}
}
private static String csvEscape(String s) {
    if (s==null) return "";
    if (s.contains(",") || s.contains("\"")) return "\"" + s.replac
e("\"", "\\\"") + "\"";
    return s;
}
}
/* =====
*                               Circular buffer
* =====
*/
class CircularBuffer122<T> {
    private final Object[] arr;
    private int head=0, size=0;
    CircularBuffer122(int capacity) {
        this.arr = new Object[Math.max(512, capacity)];
    }
    void add(T v){
        arr[head] = v;
        head = (head+1) % arr.length;
        if (size < arr.length) size++;
    }
    @SuppressWarnings("unchecked")
    List<T> asList() {
        List<T> out = new ArrayList<>(size);
        for (int i=size-1;i>=0;i--){
            int idx = (head - 1 - i + arr.length) % arr.length;
            out.add((T)arr[idx]);
        }
        return out;
    }
    Iterable<T> iterable() { return this::iterator; }
    private Iterator<T> iterator() {

```

```

        List<T> snap = asList();
        return snap.iterator();
    }
    List<T> snapshotView(){ return asList(); }
}

/* =====
 *
 * Random simulator
 * =====
 */
class VentRandomSimulator {
    private final VentSensorSuite sensors;
    VentRandomSimulator(VentSensorSuite s){this.sensors=s;}
    void runSteps(TunnelAirQualityEx ctrl, int steps){
        for (int i=0;i<steps;i++){
            ctrl.updateWithSensors(sensors.sample());
        }
    }
}

/* =====
 *
 * Swing GUI (optional)
 * =====
 */
class VentFrame extends JFrame {
    private final VentPanel panel;
    private final TunnelAirQualityEx ctrl;
    private final VentSensorSuite sensors;
    private final Timer timer;
    private volatile boolean running=true;
    VentFrame(TunnelAirQualityEx ctrl, VentSensorSuite sensors) {
        super("Tunnel Ventilation Monitor (c122)");
        this.ctrl = ctrl; this.sensors = sensors;
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setSize(980, 620);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        panel = new VentPanel();
        add(panel, BorderLayout.CENTER);
        JPanel ops = new JPanel(new FlowLayout(FlowLayout.LEFT));
        ops.setBorder(new EmptyBorder(8,8,8,8));
        JButton stepBtn = new JButton("Step");
        JButton runBtn = new JButton("Run");
        JButton stopBtn = new JButton("Stop");
        JButton exportBtn = new JButton("Export CSV");
    }

```

```

ops.add(stepBtn); ops.add(runBtn); ops.add(stopBtn); ops.add(ex
portBtn);
add(ops, BorderLayout.SOUTH);
stepBtn.addActionListener(e -> doStep());
runBtn.addActionListener(e -> running=true);
stopBtn.addActionListener(e -> running=false);
exportBtn.addActionListener(e -> {
    try {
        ctrl.exportCsv("tunnel_aq_gui_export.csv");
        JOptionPane.showMessageDialog(this, "Exported to tunnel
_aq_gui_export.csv");
    } catch (IOException ex) {
        JOptionPane.showMessageDialog(this, "Export failed: " +
ex.getMessage());
    }
});
timer = new Timer(150, e -> { if (running) doStep(); });
addWindowListener(new WindowAdapter() {
    @Override public void windowOpened(WindowEvent e) { timer.s
tart(); }
    @Override public void windowClosed(WindowEvent e) { timer.s
top(); }
});
}
private void doStep() {
    ctrl.updateWithSensors(sensors.sample());
    panel.feed(ctrl);
    panel.repaint();
}
}
class VentPanel extends JPanel {
    private final Deque<Double> coSeries = new ArrayDeque<>();
    private final Deque<Double> no2Series = new ArrayDeque<>();
    private final Deque<Double> pmSeries = new ArrayDeque<>();
    private static final int CAP=240;
    private final DecimalFormat df1 = new DecimalFormat("#0.0");
    private final DecimalFormat df2 = new DecimalFormat("#0.00");
    void feed(TunnelAirQualityEx ctrl) {
        // Visualization is synthetic; for production, expose getters t
o pull real values.
        // We approximate a plausible trend using slow random walks for
three series.
        push(coSeries, 2.0, 10.0, 0.22);
        push(no2Series, 0.3, 2.6, 0.05);

```

```

        push(pmSeries, 10.0, 120.0, 1.5);
    }
    private void push(Deque<Double> q, double lo, double hi, double sigma){
        double next = (q.isEmpty()? (lo+hi)/2.0 : q.getLast()) + Thread
LocalRandom.current().nextGaussian()*sigma;
        next = Math.max(lo, Math.min(hi, next));
        if (q.size()>=CAP) q.pollFirst();
        q.addLast(next);
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        // background
        g.setColor(new Color(245,247,250));
        g.fillRect(0,0,getWidth(),getHeight());
        // grid
        g.setColor(new Color(223,228,235));
        for (int x=60;x<getWidth()-
20;x+=50) g.drawLine(x,20,x,getHeight()-60);
        for (int y=20;y<getHeight()-
60;y+=40) g.drawLine(60,y,getWidth()-20,y);
        drawSeries(g, coSeries, 60, getHeight()-60, getWidth()-
100, getHeight()-120,
            new Color(72,120,232), "CO (ppm)", df2);
        drawSeries(g, no2Series, 60, getHeight()-60, getWidth()-
100, getHeight()-120,
            new Color(22,165,132), "NO2 (ppm)", df2);
        drawSeries(g, pmSeries, 60, getHeight()-60, getWidth()-
100, getHeight()-120,
            new Color(235,132,52), "PM2.5 (ug/m3)", df1);
    }
    private void drawSeries(Graphics g, Deque<Double> q, int ox, int oy
, int w, int h, Color c, String label, DecimalFormat df) {
        if (q.isEmpty()) return;
        double min = Collections.min(q), max = Collections.max(q);
        double span = Math.max(1e-6, max-min);
        g.setColor(c);
        int i=0, px=-1, py=-1;
        for (double v : q) {
            int x = ox + (int)((i*1.0/(CAP-1))*w);
            int y = oy - (int)((v - min)/span)*h);
            if (px>=0) g.drawLine(px,py,x,y);
            px=x; py=y; i++;
        }
    }

```

```

    }
    g.setColor(new Color(60,60,70));
    g.drawString(label + " min=" + df.format(min) + " max=" + df.f
ormat(max), ox, oy+20+(int)(Math.random()*15));
    }
}
/* =====
*
* Test suites
* =====
*/
class VentTestSuites {
    static void runVT(TunnelAirQualityEx s) { for (double[] d: VT_SEQ)
s.updateAirQuality(d[0],d[1],d[2]); }
    static void runZ3(TunnelAirQualityEx s) { for (double[] d: Z3_SEQ)
s.updateAirQuality(d[0],d[1],d[2]); }
    static void runUVT(TunnelAirQualityEx s) { for (double[] d: UVT_SEQ
) s.updateAirQuality(d[0],d[1],d[2]); }
    static void runFT(TunnelAirQualityEx s) { for (double[] d: FT_SEQ)
s.updateAirQuality(d[0],d[1],d[2]); }
    // --- Sequences (ported from your message; grouped as triples) ---
    // VT (first block) - kept here but not enabled by default
    static final double[][] VT_SEQ = new double[][]{
        {59.8,199.1,33.1},{-81.9,-146.6,173.9},{-
39.5,168.9,61.8},{174.2,-129.6,59.6},
        {152.9,39.0,-35.6},{-157.9,-155.5,-15.5},{154.8,104.6,-
171.0},{-109.7,44.5,-2.2},
        {127.2,180.5,-199.3},{160.3,-52.7,185.5},{-182.9,125.8,-
90.5},{66.6,48.7,-183.5},
        {-169.0,165.0,-59.3},{-175.8,-163.2,110.6},{-81.0,-
146.6,195.7},{-13.2,19.7,16.8},
        {-181.6,111.0,61.1},{178.0,172.2,45.1},{-
123.1,56.1,22.3},{-127.8,9.3,-64.0},
        {194.8,-152.6,-67.5},{52.0,-165.5,97.0},{195.5,-57.9,-
68.8},{-148.5,-172.8,176.2},
        {108.4,-0.2,3.3},{-62.9,-149.2,106.6},{52.9,22.6,-25.3},{-
26.7,-133.3,-61.9},
        {-187.0,146.9,-197.6},{131.7,-52.9,-196.5},{-22.5,114.2,-
122.4},{72.7,-30.3,-11.9},
        {-184.3,-7.0,-33.8},{11.8,62.8,120.7},{-48.7,145.8,-
102.1},{-32.0,2.4,-118.0},
        {-171.9,-72.0,30.9},{-161.4,136.9,-16.2},{-
5.3,92.8,42.3},{-132.9,95.0,-25.8},
        {-127.2,-21.8,143.8},{141.0,136.2,-62.6},{71.8,-
109.0,148.1},{196.7,-25.5,-169.4},

```

```

        {91.8,126.3,-159.2},{-4.5,89.1,40.9},{11.4,-184.3,24.3},{-
57.1,152.3,33.8},
        {196.5,-199.0,113.6},{47.6,69.5,132.2},{-114.3,-
58.3,154.6},{-17.5,-48.2,-67.8},
        {133.7,35.1,162.4},{-193.2,-74.1,-184.6},{85.4,-99.1,-
187.4},{146.0,26.6,-111.0},
        {71.3,-147.4,-146.2},{27.2,65.1,-31.3},{-112.9,0.9,-8.9},{-
28.8,-68.9,13.9},
        {2.7,144.9,-162.5},{43.5,-88.6,96.8},{-117.8,12.5,-51.5},{-
161.7,-113.5,-25.6},
        {159.0,-183.5,-199.5},{-86.0,125.5,84.9},{-120.8,-
2.4,36.2},{117.5,135.5,-106.3},
        {-41.3,-190.5,-21.3},{-123.9,-52.7,143.2},{47.6,-
9.3,78.4},{113.7,73.1,-35.1},
        {66.3,65.1,82.7},{-106.4,-66.3,-
58.9},{44.9,107.5,143.6},{12.0,-126.1,2.2},
        {113.9,41.1,-28.1},{-102.4,-149.6,162.2},{163.5,-
124.4,0.7},{52.2,-31.2,-100.2},
        {-176.4,-186.9,-124.4},{-
98.4,17.8,127.0},{56.1,94.4,9.9},{152.4,-181.1,180.1},
        {37.7,13.8,-52.7},{176.3,-195.0,-163.2},{-43.9,-171.2,-
41.8},{-66.1,-113.0,-76.4},
        {61.5,-3.1,131.7},{-149.9,-59.7,-87.1},{-50.3,-
150.1,118.0},{23.7,-137.3,172.4},
        {-164.7,-77.6,-81.8},{-176.9,192.2,158.7},{-74.5,-190.2,-
119.2},{150.2,-187.3,-188.4},
        {-138.2,-168.1,-132.6},{28.6,-147.3,164.0},{25.3,-
178.9,55.8},{114.8,-124.1,-147.3}
    };
    // FT (edge/magnitude stress; sanitized to avoid NaN/Inf in doubles
)
    static final double[][] FT_SEQ = new double[][]{
        {-
1.0E308,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0
},
        {-1.0E308,0.0,0.0},{-
1.0E308,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},
        {0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{-
1.0E308,0.0,0.0},
        {-1.0E308,0.0,0.0},{-
1.0E308,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},
        {0.0,1.0E308,1.0E308},{0.0,1.0E308,1.0E308},{0.0,1.0E308,1.
0E308},{0.0,0.0,0.0},

```

```

        {-1.0E308,-1.0E308,0.0},{-1.0E308,-1.0E308,0.0},{-1.0E308,-
1.0E308,0.0},
        {-1.0E308,-1.0E308,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{-
1.0E308,0.0,0.0},
        {0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{-
1.0E308,0.0,0.0},
        {1.0E308,1.0E308,1.0E308},{1.0E308,1.0E308,1.0E308},{0.0,0.
0,0.0},{0.0,0.0,0.0},
        {-
1.0E308,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0
},
        {0.0,0.0,0.0},{-2.2E-308,0.0,0.0},{-2.2E-308,0.0,0.0},{-
1.0E308,0.0,0.0},
        {-1.0E308,0.0,0.0},{-1.0E308,0.0,0.0},{-1.0E308,0.0,0.0},{-
1.0E308,0.0,0.0},
        {-1.0E308,0.0,0.0},{0.0,-1.0E308,0.0},{0.0,-
1.0E308,0.0},{0.0,0.0,0.0},
        {0.0,0.0,0.0},{-2.2E-308,0.0,0.0},{-0.0,0.0,0.0},{-
0.0,0.0,0.0},{-0.0,0.0,0.0},
        {-
0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},
        {0.0,0.0,0.0},{0.0,0.0,0.0},{-
1.0E308,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},
        {0.0,0.0,0.0},{-
0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},{0.0,-9.1E306,-9.1E306},
        {0.0,0.0,0.0},{0.0,0.0,0.0},{-1.0E308,0.0,0.0},{-
1.0E308,0.0,0.0},
        {-1.0E308,0.0,0.0},{0.0,0.0,0.0},{-
1.0E308,0.0,0.0},{1.0E308,1.0E308,1.0E308},
        {1.0E308,1.0E308,1.0E308},{1.0E308,1.0E308,1.0E308},{-
1.0E308,0.0,0.0},
        {-6.2E307,-1.5E308,-1.5E308},{-6.2E307,-1.5E308,-
1.5E308},{-6.2E307,-1.5E308,-1.5E308},
        {-6.2E307,-1.5E308,-1.5E308},{0.0,0.0,0.0},{-
1.0E308,0.0,0.0},{-1.0E308,0.0,0.0},
        {-1.0E308,0.0,0.0},{-
1.0E308,0.0,0.0},{0.0,0.0,0.0},{0.0,0.0,0.0},
        {-1.0E308,0.0,0.0},{-1.0E308,0.0,0.0}
    };
    // Z3 (subset)
    static final double[][] Z3_SEQ = new double[][]{
        {59.8,199.1,33.1},{-81.9,-146.6,173.9},{-
39.5,168.9,61.8},{174.2,-129.6,59.6},

```

{152.9, 39.0, -35.6}, {-157.9, -155.5, -15.5}, {154.8, 104.6, -171.0}, {-109.7, 44.5, -2.2},
{127.2, 180.5, -199.3}, {160.3, -52.7, 185.5}, {-182.9, 125.8, -90.5}, {66.6, 48.7, -183.5},
{-169.0, 165.0, -59.3}, {-175.8, -163.2, 110.6}, {-81.0, -146.6, 195.7}, {-13.2, 19.7, 16.8},
{-181.6, 111.0, 61.1}, {178.0, 172.2, 45.1}, {-123.1, 56.1, 22.3}, {-127.8, 9.3, -64.0},
{194.8, -152.6, -67.5}, {52.0, -165.5, 97.0}, {195.5, -57.9, -68.8}, {-148.5, -172.8, 176.2},
{108.4, -0.2, 3.3}, {-62.9, -149.2, 106.6}, {52.9, 22.6, -25.3}, {-26.7, -133.3, -61.9},
{-187.0, 146.9, -197.6}, {131.7, -52.9, -196.5}, {-22.5, 114.2, -122.4}, {72.7, -30.3, -11.9},
{-184.3, -7.0, -33.8}, {11.8, 62.8, 120.7}, {-48.7, 145.8, -102.1}, {-32.0, 2.4, -118.0},
{-171.9, -72.0, 30.9}, {-161.4, 136.9, -16.2}, {-5.3, 92.8, 42.3}, {-132.9, 95.0, -25.8},
{-127.2, -21.8, 143.8}, {141.0, 136.2, -62.6}, {71.8, -109.0, 148.1}, {196.7, -25.5, -169.4},
{91.8, 126.3, -159.2}, {-4.5, 89.1, 40.9}, {11.4, -184.3, 24.3}, {-57.1, 152.3, 33.8},
{196.5, -199.0, 113.6}, {47.6, 69.5, 132.2}, {-114.3, -58.3, 154.6}, {-17.5, -48.2, -67.8},
{133.7, 35.1, 162.4}, {-193.2, -74.1, -184.6}, {85.4, -99.1, -187.4}, {146.0, 26.6, -111.0},
{71.3, -147.4, -146.2}, {27.2, 65.1, -31.3}, {-112.9, 0.9, -8.9}, {-28.8, -68.9, 13.9},
{2.7, 144.9, -162.5}, {43.5, -88.6, 96.8}, {-117.8, 12.5, -51.5}, {-161.7, -113.5, -25.6},
{159.0, -183.5, -199.5}, {-86.0, 125.5, 84.9}, {-120.8, -2.4, 36.2}, {117.5, 135.5, -106.3},
{-41.3, -190.5, -21.3}, {-123.9, -52.7, 143.2}, {47.6, -9.3, 78.4}, {113.7, 73.1, -35.1},
{66.3, 65.1, 82.7}, {-106.4, -66.3, -58.9}, {44.9, 107.5, 143.6}, {12.0, -126.1, 2.2},
{113.9, 41.1, -28.1}, {-102.4, -149.6, 162.2}, {163.5, -124.4, 0.7}, {52.2, -31.2, -100.2},
{-176.4, -186.9, -124.4}, {-98.4, 17.8, 127.0}, {56.1, 94.4, 9.9}, {152.4, -181.1, 180.1},
{37.7, 13.8, -52.7}, {176.3, -195.0, -163.2}, {-43.9, -171.2, -41.8}, {-66.1, -113.0, -76.4},
{61.5, -3.1, 131.7}, {-149.9, -59.7, -87.1}, {-50.3, -150.1, 118.0}, {23.7, -137.3, 172.4},

```

        {-164.7,-77.6,-81.8},{-176.9,192.2,158.7},{-74.5,-190.2,-
119.2},{150.2,-187.3,-188.4},
        {-138.2,-168.1,-132.6},{28.6,-147.3,164.0},{25.3,-
178.9,55.8},{114.8,-124.1,-147.3}
    };
    // UVT (your main sequence - enabled by default)
    static final double[][] UVT_SEQ = {
        {59.8,199.1,33.1},{-
81.9,-146.6,173.9},{-39.5,168.9,61.8},{174.2,-129.6,59.6},
        {152.9,39.0,-35.6},{-157.9,-155.5,-15.5},{154.8,104.6,-
171.0},{-109.7,44.5,-2.2},
        {127.2,180.5,-199.3},{160.3,-52.7,185.5},{-182.9,125.8,-
90.5},{66.6,48.7,-183.5},
        {-169.0,165.0,-59.3},{-175.8,-163.2,110.6},{-81.0,-
146.6,195.7},{-13.2,19.7,16.8},
        {-181.6,111.0,61.1},{178.0,172.2,45.1},{-
123.1,56.1,22.3},{-127.8,9.3,-64.0},
        {194.8,-152.6,-67.5},{52.0,-165.5,97.0},{195.5,-57.9,-
68.8},{-148.5,-172.8,176.2},
        {108.4,-0.2,3.3},{-62.9,-149.2,106.6},{52.9,22.6,-25.3},{-
26.7,-133.3,-61.9},
        {-187.0,146.9,-197.6},{131.7,-52.9,-196.5},{-22.5,114.2,-
122.4},{72.7,-30.3,-11.9},
        {-184.3,-7.0,-33.8},{11.8,62.8,120.7},{-48.7,145.8,-
102.1},{-32.0,2.4,-118.0},
        {-171.9,-72.0,30.9},{-161.4,136.9,-16.2},{-
5.3,92.8,42.3},{-132.9,95.0,-25.8},
        {-127.2,-21.8,143.8},{141.0,136.2,-62.6},{71.8,-
109.0,148.1},{196.7,-25.5,-169.4},
        {91.8,126.3,-159.2},{-4.5,89.1,40.9},{11.4,-184.3,24.3},{-
57.1,152.3,33.8},
        {196.5,-199.0,113.6},{47.6,69.5,132.2},{-114.3,-
58.3,154.6},{-17.5,-48.2,-67.8},
        {133.7,35.1,162.4},{-193.2,-74.1,-184.6},{85.4,-99.1,-
187.4},{146.0,26.6,-111.0},
        {71.3,-147.4,-146.2},{27.2,65.1,-31.3},{-112.9,0.9,-8.9},{-
28.8,-68.9,13.9},
        {2.7,144.9,-162.5},{43.5,-88.6,96.8},{-117.8,12.5,-51.5},{-
161.7,-113.5,-25.6},
        {159.0,-183.5,-199.5},{-86.0,125.5,84.9},{-120.8,-
2.4,36.2},{117.5,135.5,-106.3},
        {-41.3,-190.5,-21.3},{-123.9,-52.7,143.2},{47.6,-
9.3,78.4},{113.7,73.1,-35.1},
        {66.3,65.1,82.7},{-106.4,-66.3,-
58.9},{44.9,107.5,143.6},{12.0,-126.1,2.2},

```

```

        {113.9,41.1,-28.1},{-102.4,-149.6,162.2},{163.5,-
124.4,0.7},{52.2,-31.2,-100.2},
        {-176.4,-186.9,-124.4},{-
98.4,17.8,127.0},{56.1,94.4,9.9},{152.4,-181.1,180.1},
        {37.7,13.8,-52.7},{176.3,-195.0,-163.2},{-43.9,-171.2,-
41.8},{-66.1,-113.0,-76.4},
        {61.5,-3.1,131.7},{-149.9,-59.7,-87.1},{-50.3,-
150.1,118.0},{23.7,-137.3,172.4},
        {-164.7,-77.6,-81.8},{-176.9,192.2,158.7},{-74.5,-190.2,-
119.2},{150.2,-187.3,-188.4},
        {-138.2,-168.1,-132.6},{28.6,-147.3,164.0},{25.3,-
178.9,55.8},{114.8,-124.1,-147.3}
    };
}

```

- Coverage report

1. Test case number: 100
2. V-Method testing branch coverage: 64.1%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c122_TunnelVentilationSystemEx.java	60.1 %	131	87	218
> VentTestSuites	25.0 %	2	6	8
> IncidentSensor	50.0 %	4	4	8
> PowerGridSensor	50.0 %	2	2	4
> VentSensorSuite	50.0 %	4	4	8
> TunnelAirQualityEx	64.1 %	59	33	92
> CircularBuffer122<T>	75.0 %	3	1	4
> VentAnomalyDetector	78.8 %	52	14	66
> VentPolicy	83.3 %	5	1	6
> AQRecord		0	0	0
> c122_TunnelVentilationSystemEx		0	0	0
> FanState		0	0	0
> GridState		0	0	0
> IncidentLevel		0	0	0
> MeteoSensor		0	0	0

3. Vibration testing instruction coverage: 89.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c122_TunnelVentilationSystemEx.java	87.4 %	6,205	894	7,099
> IncidentSensor	56.1 %	46	36	82
> VentSensorSuite	76.0 %	158	50	208
> VentAnomalyDetector	88.8 %	357	45	402
> TunnelAirQualityEx	89.3 %	762	91	853
> PowerGridSensor	89.5 %	34	4	38
> VentStats	93.8 %	301	20	321
> c122_TunnelVentilationSystemEx	96.3 %	236	9	245
> VentTestSuites	97.6 %	3,593	87	3,680
> AQRecord	100.0 %	45	0	45
> CircularBuffer122<T>	100.0 %	95	0	95
> FanState	100.0 %	54	0	54
> GridState	100.0 %	34	0	34
> IncidentLevel	100.0 %	34	0	34
> MeteoSensor	100.0 %	64	0	64

4. Fuzz testing branch coverage: 63%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c122_TunnelVentilationSystemEx.java		62.4 %	136	82	218
> VentTestSuites		25.0 %	2	6	8
> IncidentSensor		50.0 %	4	4	8
> PowerGridSensor		50.0 %	2	2	4
> VentSensorSuite		50.0 %	4	4	8
> TunnelAirQualityEx		63.0 %	58	34	92
> CircularBuffer122<T>		75.0 %	3	1	4
> VentStats		75.0 %	3	1	4
> VentAnomalyDetector		81.8 %	54	12	66
> VentPolicy		100.0 %	6	0	6
> AQRecord			0	0	0
> c122_TunnelVentilationSystemEx			0	0	0
> FanState			0	0	0
> GridState			0	0	0
> IncidentLevel			0	0	0

5. Fuzz testing instruction coverage: 81.8%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c122_TunnelVentilationSystemEx.java		81.9 %	5,703	1,264	6,967
> IncidentSensor		56.1 %	46	36	82
> VentSensorSuite		76.0 %	158	50	208
> TunnelAirQualityEx		81.8 %	698	155	853
> VentAnomalyDetector		88.1 %	354	48	402
> PowerGridSensor		89.5 %	34	4	38
> c122_TunnelVentilationSystemEx		92.0 %	104	9	113
> VentTestSuites		97.6 %	3,593	87	3,680
> VentPolicy		98.2 %	109	2	111
> AQRecord		100.0 %	45	0	45
> CircularBuffer122<T>		100.0 %	95	0	95
> FanState		100.0 %	54	0	54
> GridState		100.0 %	34	0	34
> IncidentLevel		100.0 %	34	0	34
> MeteoSensor		100.0 %	64	0	64

6. Z3 branch coverage: 59.8%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c122_TunnelVentilationSystemEx.java		59.6 %	130	88	218
> VentTestSuites		25.0 %	2	6	8
> IncidentSensor		50.0 %	4	4	8
> PowerGridSensor		50.0 %	2	2	4
> VentSensorSuite		50.0 %	4	4	8
> TunnelAirQualityEx		59.8 %	55	37	92
> CircularBuffer122<T>		75.0 %	3	1	4
> VentAnomalyDetector		81.8 %	54	12	66
> VentPolicy		100.0 %	6	0	6
> AQRecord			0	0	0
> c122_TunnelVentilationSystemEx			0	0	0
> FanState			0	0	0
> GridState			0	0	0
> IncidentLevel			0	0	0
> MeteoSensor			0	0	0

7. Z3 instruction coverage: 80.9%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c122_TunnelVentilationSystemEx.java		87.6 %	8,931	1,267	10,198
> IncidentSensor		56.1 %	46	36	82
> VentSensorSuite		76.0 %	158	50	208
> TunnelAirQualityEx		80.9 %	690	163	853
> VentAnomalyDetector		88.8 %	357	45	402
> PowerGridSensor		89.5 %	34	4	38
> c122_TunnelVentilationSystemEx		92.0 %	104	9	113
> VentTestSuites		98.7 %	6,824	87	6,911
> AQRecord		100.0 %	45	0	45
> CircularBuffer122<T>		100.0 %	95	0	95
> FanState		100.0 %	54	0	54
> GridState		100.0 %	34	0	34
> IncidentLevel		100.0 %	34	0	34
> MeteoSensor		100.0 %	64	0	64
> PortalWindSensor		100.0 %	44	0	44

8. Unimproved V-method branch coverage: 59.8%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c122_TunnelVentilationSystemEx.java		59.6 %	130	88	218
> VentTestSuites		25.0 %	2	6	8
> IncidentSensor		50.0 %	4	4	8
> PowerGridSensor		50.0 %	2	2	4
> VentSensorSuite		50.0 %	4	4	8
> TunnelAirQualityEx		59.8 %	55	37	92
> CircularBuffer122<T>		75.0 %	3	1	4
> VentAnomalyDetector		81.8 %	54	12	66
> VentPolicy		100.0 %	6	0	6
> AQRecord			0	0	0
> c122_TunnelVentilationSystemEx			0	0	0
> FanState			0	0	0
> GridState			0	0	0
> IncidentLevel			0	0	0
> MeteoSensor			0	0	0

9. Unimproved V-method instruction coverage: 80.9%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c122_TunnelVentilationSystemEx.java		87.6 %	8,931	1,267	10,198
> IncidentSensor		56.1 %	46	36	82
> VentSensorSuite		76.0 %	158	50	208
> TunnelAirQualityEx		80.9 %	690	163	853
> VentAnomalyDetector		88.8 %	357	45	402
> PowerGridSensor		89.5 %	34	4	38
> c122_TunnelVentilationSystemEx		92.0 %	104	9	113
> VentTestSuites		98.7 %	6,824	87	6,911
> AQRecord		100.0 %	45	0	45
> CircularBuffer122<T>		100.0 %	95	0	95
> FanState		100.0 %	54	0	54
> GridState		100.0 %	34	0	34
> IncidentLevel		100.0 %	34	0	34
> MeteoSensor		100.0 %	64	0	64
> PortalWindSensor		100.0 %	44	0	44